

Udiddit, a social news aggregator

Introduction

Udiddit, a social news aggregation, web content rating, and discussion website, is currently using a risky and unreliable Postgres database schema to store the forum posts, discussions, and votes made by their users about different topics.

The schema allows posts to be created by registered users on certain topics, and can include a URL or a text content. It also allows registered users to cast an upvote (like) or downvote (dislike) for any forum post that has been created. In addition to this, the schema also allows registered users to add comments on posts.

Here is the DDL used to create the schema:

```
CREATE TABLE bad_posts (  
    id SERIAL PRIMARY KEY,  
    topic VARCHAR(50),  
    username VARCHAR(50),  
    title VARCHAR(150),  
    url VARCHAR(4000) DEFAULT NULL,  
    text_content TEXT DEFAULT NULL,  
    upvotes TEXT,  
    downvotes TEXT  
);  
  
CREATE TABLE bad_comments (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50),  
    post_id BIGINT,  
    text_content TEXT  
);
```

Part I: Investigate the existing schema

As a first step, investigate this schema and some of the sample data in the project's SQL workspace. Then, in your own words, outline three (3) specific things that could

be improved about this schema. Don't hesitate to outline more if you want to stand out!

- The tables are not normalized, there are no relations between the tables.
- In the "bad_comments" table, the field username is not set to not null which means we could have comments that does not belongs to any users.
- In the "bad_posts" table, the "upvote" and "downvote" are defined as TEXT even though it will be much simpler to convert them to INTEGER or SMALLINT.

Part II: Create the DDL for your new schema

Having done this initial investigation and assessment, your next goal is to dive deep into the heart of the problem and create a new schema for Udiddit. Your new schema should at least reflect fixes to the shortcomings you pointed to in the previous exercise. To help you create the new schema, a few guidelines are provided to you:

1. Guideline #1: here is a list of features and specifications that Udiddit needs in order to support its website and administrative interface:
 - a. Allow new users to register:
 - i. Each username has to be unique
 - ii. Usernames can be composed of at most 25 characters
 - iii. Usernames can't be empty
 - iv. We won't worry about user passwords for this project
 - b. Allow registered users to create new topics:
 - i. Topic names have to be unique.
 - ii. The topic's name is at most 30 characters
 - iii. The topic's name can't be empty
 - iv. Topics can have an optional description of at most 500 characters.
 - c. Allow registered users to create new posts on existing topics:
 - i. Posts have a required title of at most 100 characters
 - ii. The title of a post can't be empty.
 - iii. Posts should contain either a URL or a text content, **but not both**.
 - iv. If a topic gets deleted, all the posts associated with it should be automatically deleted too.
 - v. If the user who created the post gets deleted, then the post will remain, but it will become dissociated from that user.
 - d. Allow registered users to comment on existing posts:
 - i. A comment's text content can't be empty.
 - ii. Contrary to the current linear comments, the new structure should allow comment threads at arbitrary levels.
 - iii. If a post gets deleted, all comments associated with it should be automatically deleted too.
 - iv. If the user who created the comment gets deleted, then the comment will remain, but it will become dissociated from that user.

- v. If a comment gets deleted, then all its descendants in the thread structure should be automatically deleted too.
 - e. Make sure that a given user can only vote once on a given post:
 - i. Hint: you can store the (up/down) value of the vote as the values 1 and -1 respectively.
 - ii. If the user who cast a vote gets deleted, then all their votes will remain, but will become dissociated from the user.
 - iii. If a post gets deleted, then all the votes for that post should be automatically deleted too.
- 2. Guideline #2: here is a list of queries that Udiddit needs in order to support its website and administrative interface. Note that you don't need to produce the DQL for those queries: they are only provided to guide the design of your new database schema.
 - a. List all users who haven't logged in in the last year.
 - b. List all users who haven't created any post.
 - c. Find a user by their username.
 - d. List all topics that don't have any posts.
 - e. Find a topic by its name.
 - f. List the latest 20 posts for a given topic.
 - g. List the latest 20 posts made by a given user.
 - h. Find all posts that link to a specific URL, for moderation purposes.
 - i. List all the top-level comments (those that don't have a parent comment) for a given post.
 - j. List all the direct children of a parent comment.
 - k. List the latest 20 comments made by a given user.
 - l. Compute the score of a post, defined as the difference between the number of upvotes and the number of downvotes
- 3. Guideline #3: you'll need to use normalization, various constraints, as well as indexes in your new database schema. You should use named constraints and indexes to make your schema cleaner.
- 4. Guideline #4: your new database schema will be composed of five (5) tables that should have an auto-incrementing id as their primary key.

Once you've taken the time to think about your new schema, write the DDL for it in the space provided here:

```

/*
Creating the users table
*/
CREATE TABLE "users" (
    "id" SERIAL PRIMARY KEY,
    "username" VARCHAR(25) UNIQUE NOT NULL,
    "signup_date" TIMESTAMP DEFAULT current_timestamp,

    CONSTRAINT "length not zero" CHECK (LENGTH(TRIM("username")) > 0),
    "login_timestamp" TIMESTAMP
);
-----

/*
Creating the topics table
*/
CREATE TABLE "topics" (
    "id" SERIAL PRIMARY KEY,
    "topic_name" VARCHAR(30) UNIQUE NOT NULL,
    "description" VARCHAR(500),
    "user_id" INTEGER,

    CONSTRAINT "length not empty" CHECK (LENGTH(TRIM("topic_name")) > 0),
    CONSTRAINT "fk_user_id" FOREIGN KEY ("user_id") REFERENCES "users" ("id") ON DELETE SET NULL
);
-----

/*
Creating the posts table
*/
CREATE TABLE "posts" (
    "id" SERIAL PRIMARY KEY,
    "title" VARCHAR(100) NOT NULL,
    "url" TEXT,
    "text_content" TEXT,
    "user_id" INTEGER,
    "topic_id" INTEGER NOT NULL,

```

```

"post_date" TIMESTAMP DEFAULT current_timestamp,

CONSTRAINT "title length not empty" CHECK (LENGTH(TRIM("title")) > 0),
CONSTRAINT "fk_user_id" FOREIGN KEY ("user_id") REFERENCES "users"("id") ON DELETE SET NULL ,
CONSTRAINT "fk_topic_id" FOREIGN KEY ("topic_id") REFERENCES "topics" ("id") ON DELETE CASCADE,
CONSTRAINT "either url or text" CHECK (((("url") IS NULL AND ("text_content") IS NOT NULL) OR (("url") IS
NOT NULL AND ("text_content") IS NULL))
);

-----

/*
Creating the comments table
*/
CREATE TABLE "comments"(
    "id" SERIAL PRIMARY KEY,
    "comment" TEXT NOT NULL,
    "user_id" INTEGER,
    "post_id" INTEGER NOT NULL,
    "parent_comment_id" INTEGER DEFAULT NULL,

    CONSTRAINT "comment not empty" CHECK (LENGTH(TRIM("comment"))>0),
    CONSTRAINT "fk_user_id" FOREIGN KEY ("user_id") REFERENCES "users" ("id") ON DELETE SET NULL,
    CONSTRAINT "fk_post_id" FOREIGN KEY ("post_id") REFERENCES "posts" ("id") ON DELETE CASCADE,
    CONSTRAINT "comment thread" FOREIGN KEY ("parent_comment_id") REFERENCES "comments" ("id") ON
DELETE CASCADE
);

-----

/*
Creating the vote table
*/
CREATE TABLE "post_votes"(
    "id" SERIAL PRIMARY KEY,
    "post_vote" INTEGER NOT NULL ,
    "post_id" INTEGER NOT NULL,
    "user_id" INTEGER,

    CONSTRAINT "vote_values" CHECK ("post_vote" =1 OR "post_vote" = -1 ),

```

```
CONSTRAINT "fk_post_id" FOREIGN KEY ("post_id") REFERENCES "posts" ("id") ON DELETE CASCADE,  
CONSTRAINT "fk_user_id" FOREIGN KEY ("user_id") REFERENCES "users" ("id") ON DELETE SET NULL,  
CONSTRAINT "unique_user_post" UNIQUE ("user_id","post_id")  
);
```

```
-----  
CREATE INDEX "find_users_not_logged_in_last_year" ON "users" ("username","login_timestamp");  
CREATE INDEX "users_not_logged_in_last_year" ON "users" (LOWER("username"),"signup_date");  
CREATE INDEX "find_user_ids_in_posts" ON "posts" ("user_id");  
CREATE INDEX "topics_does_not_have_any_posts" ON "posts" ("topic_id");  
CREATE INDEX "find_posts_by_title" ON "posts" (LOWER("title") VARCHAR_PATTERN_OPS);  
CREATE INDEX "find_posts_by_title" ON "posts" ("url");  
CREATE INDEX "find_posts_by_topic_and_date" ON "posts" ("topic_id", "post_date");  
CREATE INDEX "find_direct_children_of_comment" ON "comments" ("comment","parent_comment_id");  
CREATE INDEX "comment_by_user" ON "comments" ("user_id");  
CREATE INDEX "score_of_post" ON "post_votes" ("post_vote","post_id");
```

Part III: Migrate the provided data

Now that your new schema is created, it's time to migrate the data from the provided schema in the project's SQL Workspace to your own schema. This will allow you to review some DML and DQL concepts, as you'll be using INSERT...SELECT queries to do so. Here are a few guidelines to help you in this process:

1. Topic descriptions can all be empty
2. Since the bad_comments table doesn't have the threading feature, you can migrate all comments as top-level comments, i.e. without a parent
3. You can use the Postgres string function **regexp_split_to_table** to unwind the comma-separated votes values into separate rows
4. Don't forget that some users only vote or comment, and haven't created any posts. You'll have to create those users too.
5. The order of your migrations matter! For example, since posts depend on users and topics, you'll have to migrate the latter first.
6. Tip: You can start by running only SELECTs to fine-tune your queries, and use a LIMIT to avoid large data sets. Once you know you have the correct query, you can then run your full INSERT...SELECT query.
7. **NOTE:** The data in your SQL Workspace contains thousands of posts and comments. The DML queries may take at least 10-15 seconds to run.

Write the DML to migrate the current data in bad_posts and bad_comments to your new database schema:

```
-- Migrate topic name
INSERT INTO "topics" ("topic_name") SELECT DISTINCT "topic" FROM "bad_posts";

-- Migrate Usernames

INSERT INTO "users" ("username")
SELECT "username" FROM "bad_posts"
UNION
SELECT "username" FROM "bad_comments"
UNION
```



```
SELECT regexp_split_to_table("upvotes",',') FROM "bad_posts"
UNION
SELECT regexp_split_to_table("upvotes",',') FROM "bad_posts"
-- Adding user id and topic id coloumn to bad_posts table

ALTER TABLE "bad_posts"
ADD COLUMN "user_id" INTEGER,
ADD COLUMN "topic_id" INTEGER;

-- update the value of user_id joining the value from the users table

UPDATE "bad_posts" SET "user_id" = (
SELECT "id"
FROM "users"
WHERE "users"."username" = "bad_posts"."username"
);

-- update the value of topic_id, joining the value from the topics table

UPDATE "bad_posts" SET "topic_id"=(
SELECT "id"
FROM "topics"
WHERE "topics"."topic_name"="bad_posts"."topic"
);

-- insert the records into the posts table

INSERT INTO "posts" ("title","url","text_content","user_id","topic_id")
SELECT LEFT("title",100),
"url",
"text_content",
"user_id",
"topic_id"
FROM "bad_posts" ;
```

```
-- Insert the records into the comments table
```

```
INSERT INTO "comments" ("comment","user_id","post_id")
SELECT "bc"."text_content",
"po"."id",
"bc"."post_id"
FROM "bad_comments" bc
JOIN "users" us
ON "us"."username" = "bc"."username"
JOIN "posts" po
ON "po"."id" = "bc"."post_id" ;
```

```
-- Insert the votes into the post_votes
```

```
INSERT INTO "post_votes" ("post_vote","post_id","user_id")
SELECT 1,"up"."id", "us"."id"
FROM (SELECT "id" , regexp_split_to_table("upvotes",',') AS user_list
FROM "bad_posts") AS up
LEFT JOIN "users" us
ON "us"."username" = "up"."user_list"
LEFT JOIN "posts" po
ON "po"."id" = "up"."id"
UNION ALL
SELECT -1,"up"."id", "us"."id"
FROM (SELECT "id" , regexp_split_to_table("downvotes",',') AS user_list
FROM "bad_posts") AS up
LEFT JOIN "users" us
ON "us"."username" = "up"."user_list"
LEFT JOIN "posts" po
ON "po"."id" = "up"."id" ;
```

```
-- Removing the added coulmn that helped during the migration
```

```
ALTER TABLE "bad_posts"  
DROP COLUMN "user_id" ,  
DROP COLUMN "topic_id" ;
```